

To use **IORedis** with **NodeJS**, we need to install the required packages:

```
{
  "name": "nodejs-ioredis",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs",
  "dependencies": {
    "dotenv": "^17.4.2",
    "ioredis": "^5.10.1"
  }
}
```

=====

To config the application to use Redis, we need to define the configuration in the .env file:

```
REDIS_HOST=
REDIS_PORT=
REDIS_PASSWORD=
REDIS_DB=
REDIS_TLS=
REDIS_PREFIX=
```

=====

Then to define the Redis config file for the NodeJS application:

```
const Redis = require('ioredis')
require('dotenv').config()

// Production-ready Redis configuration
const redisConfig = {
  // Connection settings
  host: process.env.REDIS_HOST || 'localhost',
  port: process.env.REDIS_PORT || 6379,
  password: process.env.REDIS_PASSWORD || undefined,
  db: process.env.REDIS_DB || 0,

  // Connection pool & timeouts
  maxRetriesPerRequest: 3,
  enableReadyCheck: true,
  maxLoadingTimeout: 30000,
```

```

// Reconnection strategy
retryStrategy(times) {
  const delay = Math.min(times * 50, 2000)
  console.log(`Redis reconnecting... attempt ${times}, delay: ${delay}ms`)
  return delay
},

// Connection timeout
connectTimeout: 10000,

// Keep alive
keepAlive: 30000,

// TLS for production (optional)
tls: process.env.REDIS_TLS === 'true' ? {
  rejectUnauthorized: false
} : undefined,

// Key prefix for namespace isolation
keyPrefix: process.env.REDIS_PREFIX || 'app:',

// Lazy connect - connect on first command
lazyConnect: false,

// Show friendly error stack
showFriendlyErrorStack: process.env.NODE_ENV !== 'production'
}

// Create Redis instance
const redis = new Redis(redisConfig)

// Event handlers for monitoring
redis.on('connect', () => {
  console.log('Redis: Connected')
})

redis.on('ready', () => {
  console.log('Redis: Ready')
})

redis.on('error', (err) => {
  console.error('Redis Error:', err.message)
})

```

```

redis.on('close', () => {
  console.log('Redis: Connection closed')
})

redis.on('reconnecting', () => {
  console.log('Redis: Reconnecting...')
})

redis.on('end', () => {
  console.log('Redis: End')
})

module.exports = redis
=====

```

To define the redis service utility class:

```

const redis = require('../config/redis')
const Redis = require('ioredis')

class RedisService {
  constructor() {
    this.redis = redis
  }
  // here we set the operations
}
-----

```

To define the strings methods:

```

// ===== STRING OPERATIONS =====

async set(key, value, ttlSeconds = null) {
  try {
    const serialized = JSON.stringify(value)

    if (ttlSeconds) {
      await this.redis.setex(key, ttlSeconds, serialized)
    } else {
      await this.redis.set(key, serialized)
    }

    return true
  } catch (error) {
    console.error(`Redis SET error for key ${key}:`, error)
    throw error
  }
}

```

```

}

async get(key) {
  try {
    const data = await this.redis.get(key)
    return data ? JSON.parse(data) : null
  } catch (error) {
    console.error(`Redis GET error for key ${key}:`, error)
    throw error
  }
}

async delete(key) {
  try {
    const result = await this.redis.del(key)
    return result > 0
  } catch (error) {
    console.error(`Redis DEL error for key ${key}:`, error)
    throw error
  }
}

async exists(key) {
  try {
    return await this.redis.exists(key)
  } catch (error) {
    console.error(`Redis EXISTS error for key ${key}:`, error)
    throw error
  }
}

async expire(key, seconds) {
  try {
    return await this.redis.expire(key, seconds)
  } catch (error) {
    console.error(`Redis EXPIRE error for key ${key}:`, error)
    throw error
  }
}

```

To define the hash methods:

```
// ===== HASH OPERATIONS =====
```

```
async hset(key, field, value) {
  try {
    const serialized = typeof value === 'object' ? JSON.stringify(value) :
value
    return await this.redis.hset(key, field, serialized)
  } catch (error) {
    console.error(`Redis HSET error:`, error)
    throw error
  }
}

async hget(key, field) {
  try {
    const data = await this.redis.hget(key, field)
    if (!data) return null

    try {
      return JSON.parse(data)
    } catch {
      return data
    }
  } catch (error) {
    console.error(`Redis HGET error:`, error)
    throw error
  }
}

async hgetall(key) {
  try {
    const data = await this.redis.hgetall(key)
    // Parse JSON values
    const result = {}
    for (const [field, value] of Object.entries(data)) {
      try {
        result[field] = JSON.parse(value)
      } catch {
        result[field] = value
      }
    }
    return result
  } catch (error) {
    console.error(`Redis HGETALL error:`, error)
  }
}
```

```

        throw error
    }
}

async hdel(key, ...fields) {
    try {
        return await this.redis.hdel(key, ...fields)
    } catch (error) {
        console.error(`Redis HDEL error:`, error)
        throw error
    }
}
}

```

To define the list methods:

// ===== LIST OPERATIONS =====

```

async lpush(key, ...values) {
    try {
        const serialized = values.map(v => JSON.stringify(v))
        return await this.redis.lpush(key, ...serialized)
    } catch (error) {
        console.error(`Redis LPUSH error:`, error)
        throw error
    }
}

async rpush(key, ...values) {
    try {
        const serialized = values.map(v => JSON.stringify(v))
        return await this.redis.rpush(key, ...serialized)
    } catch (error) {
        console.error(`Redis RPUSH error:`, error)
        throw error
    }
}

async lpop(key) {
    try {
        const data = await this.redis.lpop(key)
        return data ? JSON.parse(data) : null
    } catch (error) {
        console.error(`Redis LPOP error:`, error)
        throw error
    }
}
}

```

```

async rpop(key) {
  try {
    const data = await this.redis.rpop(key)
    return data ? JSON.parse(data) : null
  } catch (error) {
    console.error(`Redis RPOP error:`, error)
    throw error
  }
}

async lrange(key, start = 0, stop = -1) {
  try {
    const data = await this.redis.lrange(key, start, stop)
    return data.map(item => {
      try {
        return JSON.parse(item)
      } catch {
        return item
      }
    })
  } catch (error) {
    console.error(`Redis LRANGE error:`, error)
    throw error
  }
}

```

To define the Set methods:

// ===== SET OPERATIONS =====

```

async sadd(key, ...members) {
  try {
    const serialized = members.map(m => JSON.stringify(m))
    return await this.redis.sadd(key, ...serialized)
  } catch (error) {
    console.error(`Redis SADD error:`, error)
    throw error
  }
}

async smembers(key) {
  try {
    const data = await this.redis.smembers(key)
    return data.map(item => {
      try {

```

```

        return JSON.parse(item)
    } catch {
        return item
    }
  })
} catch (error) {
  console.error(`Redis SMEMBERS error:`, error)
  throw error
}
}

async srem(key, ...members) {
  try {
    const serialized = members.map(m => JSON.stringify(m))
    return await this.redis.srem(key, ...serialized)
  } catch (error) {
    console.error(`Redis SREM error:`, error)
    throw error
  }
}

async sismember(key, member) {
  try {
    return await this.redis.sismember(key, JSON.stringify(member))
  } catch (error) {
    console.error(`Redis SISEMEMBER error:`, error)
    throw error
  }
}

```

To define the sorted set methods:

// ===== SORTED SET OPERATIONS =====

```

async zadd(key, score, member) {
  try {
    const serialized = JSON.stringify(member)
    return await this.redis.zadd(key, score, serialized)
  } catch (error) {
    console.error(`Redis ZADD error:`, error)
    throw error
  }
}

```

```

async zrange(key, start = 0, stop = -1, withScores = false) {
  try {
    const args = withScores ? [start, stop, 'WITHSCORES'] : [start, stop]
    const data = await this.redis.zrange(key, ...args)

    if (!withScores) {
      return data.map(item => {
        try {
          return JSON.parse(item)
        } catch {
          return item
        }
      })
    }

    // Parse with scores
    const result = []
    for (let i = 0; i < data.length; i+=2) {
      result.push({
        member: JSON.parse(data[i]),
        score: parseFloat(data[i + 1])
      })
    }
    return result
  } catch (error) {
    console.error(`Redis ZRANGE error:`, error)
    throw error
  }
}

async zrem(key, ...members) {
  try {
    const serialized = members.map(m => JSON.stringify(m))
    return await this.redis.zrem(key, ...serialized)
  } catch (error) {
    console.error(`Redis ZREM error:`, error)
    throw error
  }
}

```

To define the Pipeline and Transaction methods:

```
// ===== PIPELINE OPERATIONS =====

/**
 * Execute multiple commands in a pipeline for better performance
 * @param {Function} pipelineFn - Function that receives pipeline instance
 * @returns {Array} - Results of all commands
 */
async pipeline(pipelineFn) {
  try {
    const pipeline = this.redis.pipeline()
    pipelineFn(pipeline)
    const results = await pipeline.exec()

    // Parse results and handle errors
    return results.map(([err, result]) => {
      if (err) throw err

      // Try to parse JSON for string results
      if (typeof result === 'string') {
        try {
          return JSON.parse(result)
        } catch {
          return result
        }
      }
      return result
    })
  } catch (error) {
    console.error('Redis Pipeline error:', error)
    throw error
  }
}

/**
 * Atomic transaction with MULTI/EXEC
 * @param {Function} transactionFn - Function that receives multi instance
 * @returns {Array} - Results of all commands
 */
async transaction(transactionFn) {
  try {
    const multi = this.redis.multi()
    transactionFn(multi)
    const results = await multi.exec()
  }
}
```

```

        return results.map(([err, result]) => {
            if (err) throw err
            return result
        })
    } catch (error) {
        console.error('Redis Transaction error:', error)
        throw error
    }
}

```

To define the Pub/Sub methods:

// ===== PUB/SUB OPERATIONS =====

```

async publish(channel, message) {
    try {
        const serialized = JSON.stringify(message)
        return await this.redis.publish(channel, serialized)
    } catch (error) {
        console.error(`Redis PUBLISH error:`, error)
        throw error
    }
}

async subscribe(channel, callback) {
    // Create separate connection for pub/sub
    const subscriber = new Redis({
        host: process.env.REDIS_HOST || 'localhost',
        port: process.env.REDIS_PORT || 6379,
        password: process.env.REDIS_PASSWORD || undefined,
        retryStrategy: (times) => Math.min(times * 50, 2000)
    })

    await subscriber.subscribe(channel)

    subscriber.on('message', (receivedChannel, message) => {
        try {
            const parsed = JSON.parse(message)
            callback(receivedChannel, parsed)
        } catch {
            callback(receivedChannel, message)
        }
    })
}

```

```

    return {
      unsubscribe: () => subscriber.unsubscribe(channel),
      quit: () => subscriber.quit()
    }
  }
}

```

To define the utility methods:

```
// ===== UTILITY OPERATIONS =====
```

```

async keys(pattern) {
  try {
    return await this.redis.keys(pattern)
  } catch (error) {
    console.error(`Redis KEYS error:`, error)
    throw error
  }
}

async flushdb() {
  try {
    // Only allow in development
    if (process.env.NODE_ENV === 'production') {
      throw new Error('FLUSHDB not allowed in production')
    }
    return await this.redis.flushdb()
  } catch (error) {
    console.error(`Redis FLUSHDB error:`, error)
    throw error
  }
}

async ping() {
  try {
    return await this.redis.ping()
  } catch (error) {
    console.error(`Redis PING error:`, error)
    throw error
  }
}

async info() {
  try {
    return await this.redis.info()
  } catch (error) {
    console.error(`Redis INFO error:`, error)
  }
}

```

```
        throw error
    }
}

// Graceful shutdown
async quit() {
    await this.redis.quit()
}

async disconnect() {
    this.redis.disconnect()
}
```